

Spark — Universal Intelligence System

Spark is a standalone, product-agnostic intelligence platform. It is **not** a chatbot, not a UI, and not tied to any single product. Products (Bud, a future Realm Assistant, a future Orbit Assistant, etc.) consume Spark's intelligence through one stable SDK boundary while keeping their own personality and purpose.

```
import { createSpark } from "@lib/spark";

const spark = createSpark();
await spark.initialize();

spark.identity.createContext({ userId: "u1", sessionId: "s1", product: "bud" });
const results = await spark.search.search("refund policy");
const plan = spark.planning.createPlan("Answer the user's question", ["Search kno
```

Consumers should only ever `import { ... } from "@lib/spark"` . **Nothing under** `spark/core/*` , `spark/memory/*` , `spark/intelligence/*` , **etc. is a supported import path** — those are internal and may change.

Status

This is **Phase 1**: architecture, interfaces, and local (offline, no API key, no network call) implementations for every module. Real hosted model providers (OpenAI, Anthropic, Gemini) are wired as placeholder adapters behind the `AIPProvider` interface but are not yet implemented — Spark runs entirely on the built-in `MockProvider` until you register a real one.

Verified in this repo:

- `npm run typecheck` — passes with strict TypeScript
- `npm run smoke-test` — exercises all 18 services end to end

Architecture

```
spark/
  container.ts      DI container (per-instance, lazy singletons)
  events.ts         Internal event bus (memory.updated, search.completed, ...)
```

middleware.ts	Middleware pipeline (logging, auth, cache, rate-limit)
plugins.ts	Plugin registration (extend Spark without modifying core)
manifest.ts	Static metadata (version, capabilities)
types.ts	Health/metrics types
index.ts	Public SDK – the ONLY supported import path
core/	
identity/	User, session, product, locale, timezone, device context
reasoning/	Analysis and decisions, with explainable steps
planning/	Goals, task sequencing, dependency-aware plans
memory/	Short-term, long-term, semantic, episodic, workspace
context/	Session/environment awareness (separate from identity)
knowledge/	Document store + naive relevance query (swap for a real knowledge graph / embeddings store later)
intelligence/	
search/	Depends only on Knowledge (Search -> Knowledge)
recommendations/	Tag-overlap scoring
organization/	Grouping / sorting
personalization/	Per-user preference store
writing/	Delegates to AIProvider
translation/	Delegates to AIProvider
summaries/	Deterministic extractive summarizer
trust/	
privacy/	PII detection + redaction (regex-based)
security/	Heuristic injection/scam-pattern scanner
automation/	Named task registration + execution + history
learning/	Feedback capture + aggregate scoring
providers/	
types.ts	AIProvider interface – the ONLY thing services call
registry.ts	Registration, default selection, availability fal
mock.ts	Default provider – deterministic, offline, alwa
openai.ts / anthropic.ts /	Placeholder adapters. No SDK dependency, no netw
gemini.ts / local.ts	calls. Implement complete() when ready to go l

Dependency direction

Services depend downward and never sideways in a way that would create cycles:

Search -> Knowledge

Writing / Translation / Reasoning -> ProviderRegistry (AIProvider interface)

Spark never imports any product (Bud, UNIBUD, My Realm, Orbit). Products only ever import `@/lib/spark`.

Lifecycle

```
await spark.initialize(); // resolves every registered service once, eagerly
await spark.reset();      // clears the container and re-registers core services
await spark.shutdown();   // clears event listeners and middleware
```

Introspection

```
spark.version();          // "0.1.0"
spark.modules();          // registered service tokens
spark.capabilities();     // ["identity", "reasoning", ..., "learning"]
spark.manifest();         // name, version, build, capabilities, modules, providers
spark.health();           // status, loaded modules, provider availability, diagnost
spark.metrics();          // memory size, events emitted (extend as real counters ar
```

Adding a real provider

```
import { createSpark, OpenAIProvider } from "@/lib/spark";

const spark = createSpark();
spark.registerProvider(new OpenAIProvider(process.env.OPENAI_API_KEY), /* makeDef
```

Until `OpenAIProvider.complete()` is implemented, `ProviderRegistry.resolve()` will keep silently falling back to `MockProvider` when the registered provider reports `isAvailable() === false` — Spark never throws just because a real provider isn't configured yet.

Extending Spark

- **Middleware** — `spark.use(loggingMiddleware)` or write your own `(ctx, next) => Promise<T>` function for auth, caching, rate limiting.
- **Plugins** — `spark.registerPlugin({ name, install(ctx) { ... } })` to add services, providers, or middleware without touching Spark's core.
- **Events** — `spark.events.on("search.completed", handler)` . Note: no service currently emits domain events yet; wire `events.emit(...)` calls into individual

services as needed.

Explicitly out of scope for this phase

- No hosted model calls (all providers besides Mock throw until implemented)
- No persistence (everything is in-memory and resets on process restart)
- No product integration (Bud/UNIBUD/My Realm/Orbit are not referenced anywhere)
- No UI of any kind